



International University of Sarajevo

Faculty of Engineering and Natural Sciences

Computer Architecture

CS304 Spring 2025

PROJECT 1 REPORT

Matrices Operations

Students: Berina Juković

Hena Šehović

Berin Žunić

Sarajevo, 09.04.2025.

Contents

1.	Introduction	3
2.	The Project	5
2.1.	MIPS Commands used	5
2.2.	Program Architecture	6
2.2.1.	Data Segment Organization	6
2.2.2.	Input Function	8
2.2.3.	Display Matrix	13
2.2.4.	Print Matrix.....	16
2.2.5.	Add Matrices.....	17
2.2.6	Subtract Matrices.....	19
2.2.7	Multiplication of matrices	20
3.	Conclusion.....	24
4.	Resources	24
5.	List of Figures.....	24

1. Introduction

This project shows the use of fundamental matrix operations with assembly language programming, demonstrating low-level implementation of mathematical operations encapsulated in high-level languages. Highlighting computer architecture principles, we have designed effective assembly procedures to perform matrix addition, subtraction, and multiplication.

Implementation emphasizes:

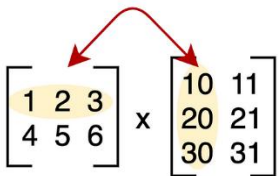
- Direct register and memory manipulation to achieve highest performance
- Optimization of use of processor facilities like parallel operations
- Effective control of the memory hierarchy (registers, cache, RAM)
- Optimization techniques specific to assembly programming

Our solution provides an overview of how modern processors execute arithmetic operations in the hardware and the architectural design issues related to handling two-dimensional data structures. Our project is a real-world examination of the connection between algorithmic design, instruction set architecture, and actual hardware implementation.

This example shows how assembly language provides fine control over computational resources while highlighting the challenges of managing complex data structures at this abstraction level. The program is interactive, as it allows users to input matrices, execute operations, and display results while ensuring proper error handling for dimension mismatches. The following figures visually show how a matrix looks and what the program does.

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

Picture 1: Matrix notation



$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$

$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

Picture 2: How matrices multiplication works

$$\begin{aligned}
 A + B &= \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 3 & 6 & 9 \\ 8 & 5 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} - \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix} \\
 &= \begin{bmatrix} 1+3 & 2+6 & 3+9 \\ 4+8 & 5+5 & 6+2 \end{bmatrix} = \begin{bmatrix} 1-2 & 2-4 \\ 3-1 & 1-3 \end{bmatrix} \\
 &= \begin{bmatrix} 4 & 8 & 12 \\ 12 & 10 & 8 \end{bmatrix} = \begin{bmatrix} -1 & -2 \\ 2 & -2 \end{bmatrix}
 \end{aligned}$$

Pictures 3 and 4: Matrices summation and subtraction

2. The Project

2.1. MIPS Commands used

This part of the reports aims to explain and list the commands used in creating this project, the commands are divided based on their purpose.

1. Data Declaration Commands

.space N -Reserves N bytes of uninitialized memory
.word val -Stores a 32-bit integer value
.ascii "str"-Stores a null-terminated string

2. Arithmetic/Logical Operations

add rd, rs, rt-Adds two registers ($rd = rs + rt$)
sub rd, rs, rt-Subtracts ($rd = rs - rt$)
mul rd, rs, rt-Multiplies ($rd = rs * rt$)

3. Memory Access Commands

lw rt, offset(base)-Loads a word from memory
sw rt, offset(base)-Stores a word to memory
la rd, label-Loads address of a label

4. Control Flow Commands

j label-Unconditional jump
beq rs, rt, label-Branches if $rs == rt$
bne rs, rt, label-Branches if $rs != rt$
jal label-Jumps and saves return address in \$ra
jr rs-Jumps to address in register

5. System Calls (Syscalls)

li \$v0, 1-Prepares to print integer
li \$v0, 4-Prepares to print string
li \$v0, 5-Reads integer input
li \$v0, 10-Terminates program
syscall-Executes the prepared syscall

6. Register Usage Conventions

\$s0-\$s7-Saved variables (preserved across calls)
\$t0-\$t9-Temporary registers

\$a0-\$a3-Argument passing
\$v0-\$v1-Return values
\$ra-Return address

2.2. Program Architecture

2.2.1. Data Segment Organization

In this part we declare variables and reserve memory or initialize all necessary variables

```
matrixA:    .space 400    # Space for matrix A (up to 10x10)
matrixB:    .space 400    # Space for matrix B (up to 10x10)
matrixC:    .space 400    # Space for result matrix
```

Picture 5: Declaring variables and reserve bytes in memory

In this part use .space 400 to reserve 400 bytes of memory for each matrix. The reason is each 32-bit integer requires 4 bit and we want to have matrices up to 10x10 dimensions and with simple math we can conclude that each matrix requires 400 bytes of memory.

```
rowsA:      .word 0
colsA:      .word 0
rowsB:      .word 0
colsB:      .word 0
```

Picture 6: Storing values

With command .word We store a 32-bit integer values for rows and columns of matrices A and B, the reason is to prevents operations on uninitialized matrices.

```
# Messages
menu:      .asciiz "\nMatrix Operations Menu:\n1. Enter matrices\n2. Add matrices\n3. Subtract matrices\n4. Multiply matrices\n5. Display matrices\n6. Exit\nEnter choice: "
resultMsg: .asciiz "\nResult Matrix:\n"
newline:   .asciiz "\n"
space:     .asciiz " "
tab:       .asciiz "\t"
errorMsg:  .asciiz "\nError: Matrix dimensions don't match!\n"
multError: .asciiz "\nError: For multiplication, colsA must equal rowsB!\n"
enterRows: .asciiz "\nEnter number of rows (1-10): "
enterCols: .asciiz "Enter number of columns (1-10): "
enterVal:  .asciiz "Enter value for ["
closeBrack: .asciiz "]"
matrixAMsg: .asciiz "\nMatrix A:\n"
matrixBMsg: .asciiz "\nMatrix B:\n"
currentDims: .asciiz "\nCurrent dimensions:\nMatrix A: "
xSymbol:   .asciiz " x "
matrixBText: .asciiz "\nMatrix B: "
```

Picture 7: Messages

Also create a lot of variables as messages to make easier code for read and write. We used .asciiz to store a null-terminated string

```

.text
.globl main
main:
    j show_menu

```

Picture 8: Text segment and main function

At this part we start with our code and in the main function we see jump to show_menu function.

```

show_menu:
    # Display menu
    li $v0, 4
    la $a0, menu
    syscall

    # Get user choice
    li $v0, 5
    syscall

    # Process choice
    beq $v0, 1, input_matrices
    beq $v0, 2, add_matrices
    beq $v0, 3, subtract_matrices
    beq $v0, 4, multiply_matrices
    beq $v0, 5, display_matrices
    beq $v0, 6, exit_program

    # Invalid choice
    j show_menu

```

Picture 9: Function show_menu

As you can see this function display the menu, get user choice and process choice.

```

exit_program:
    li $v0, 10
    syscall

```

Picture 10: Function exit_program

This function just stop the execution of the code.

```

dimension_error:
    # Print error message
    li $v0, 4
    la $a0, errorMsg
    syscall
    j show_menu

multiplication_error:
    # Print specific multiplication error
    li $v0, 4
    la $a0, multError
    syscall
    j show_menu

```

Picture 11: Functions dimension_error and multiplication_error

In these two functions the only purpose is to print the message(error) and jump in show_menu function.

2.2.2. Input Function

```

# Matrix input function
input_matrices:
    # Input matrix A dimensions
    la $a0, enterRows
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    sw $v0, rowsA

    la $a0, enterCols
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    sw $v0, colsA

    # Input matrix A elements
    la $a0, matrixAMsg
    li $v0, 4
    syscall

    lw $s0, rowsA      # $s0 = rowsA
    lw $s1, colsA      # $s1 = colsA
    li $t0, 0          # row counter

```

Picture 12: Input matrix for matrix A (Part 1)

First we start with inputs for matrix A and the first step is to input the dimension. So program will print the message then take the value and store them. This procedure will

execute two times, for row and column. After that program will print the "matrixAMsg" and load value to registers to make preparation for loop.

```

input_loop_A_row:
    bge $t0, $s0, input_matrixB
    li $t1, 0          # col counter

input_loop_A_col:
    bge $t1, $s1, end_input_A_col

    # Prompt for value
    la $a0, enterVal
    li $v0, 4
    syscall
    move $a0, $t0
    li $v0, 1
    syscall
    la $a0, space
    li $v0, 4
    syscall
    move $a0, $t1
    li $v0, 1
    syscall
    la $a0, closeBrack
    li $v0, 4
    syscall

```

Picture 13: : Input matrix for matrix A (Part 2)

So the first thing in loop is to check is number of row and counter of row are the same, if it is then program will go to function input_matrixB else then it continue with filling the matrix. The next step to set column counter to 0 and then enter the second loop.

In this loop the first thing is to check is number of column and counter of column are the same if it is then jump to end_input_A_col else then prompt the value. As you can see program will first print the message "enterVal", copy row index to argument and print that argument, then print the space, do the same for column as do for row and then print closeBrack. As result we have this:

|Enter value for [0 0]:

Picture 14: Result after code execution

```

# Get value
li $v0, 5          # Read integer
syscall

# Calculate address and store
mul $t2, $t0, $s1
add $t2, $t2, $t1
sll $t2, $t2, 2
la $t3, matrixA
add $t3, $t3, $t2
sw $v0, 0($t3)

addi $t1, $t1, 1
j input_loop_A_col

end_input_A_col:
la $a0, newline
li $v0, 4
syscall

addi $t0, $t0, 1
j input_loop_A_row

```

Picture 15: Input matrix for matrix A (Part 3)

So, when we print this beautiful message, now is time to get the value and store. So first thing we get the value that we enter then according to this addressing formula we store the values: $\$t2 = (\text{row} * \text{cols} + \text{col}) * 4$ and this is the first three line of that part of code then we load base address add offset to base and store the value. Then we increment column counter and repeat inner loop.

When number of column and counter of column are equal then we start with function **end_input_A_col**. In this function we print newline, then increment counter of row and jump to function **input_loop_A_row**. This procedure will repeat until number of row and counter of row are equal then we go to function **input_matrixB**.

```

input_matrixB:
    # Input matrix B dimensions
    la $a0, enterRows
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    sw $v0, rowsB

    la $a0, enterCols
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    sw $v0, colsB

    # Input matrix B elements
    la $a0, matrixBMsg
    li $v0, 4
    syscall

    lw $s0, rowsB      # $s0 = rowsB
    lw $s1, colsB      # $s1 = colsB
    li $t0, 0          # row counter

```

Picture 16: Input matrix for matrix B (Part 1)

```

input_loop_B_row:
    bge $t0, $s0, input_done
    li $t1, 0          # col counter

    input_loop_B_col:
        bge $t1, $s1, end_input_B_col

        # Prompt for value
        la $a0, enterVal
        li $v0, 4
        syscall
        move $a0, $t0
        li $v0, 1
        syscall
        la $a0, space
        li $v0, 4
        syscall
        move $a0, $t1
        li $v0, 1
        syscall
        la $a0, closeBrack
        li $v0, 4
        syscall

```

Picture 17: Input matrix for matrix B (Part 2)

```

# Get value
li $v0, 5          # Read integer
syscall

# Calculate address and store
mul $t2, $t0, $s1
add $t2, $t2, $t1
sll $t2, $t2, 2
la $t3, matrixB
add $t3, $t3, $t2
sw $v0, 0($t3)

addi $t1, $t1, 1
j input_loop_B_col

end_input_B_col:
la $a0, newline
li $v0, 4
syscall

addi $t0, $t0, 1
j input_loop_B_row

```

Picture 18: Input matrix for matrix B (Part 3)

The function `input_matrixB`. are similar, but only difference is at first condition where if number of row and counter of row are the same then program will jump to function `input_done`.

The purpose of function **input_done** is to print one beautiful message

```

input_done:
# Show current dimensions
la $a0, currentDims
li $v0, 4
syscall

lw $a0, rowsA
li $v0, 1
syscall

la $a0, xSymbol
li $v0, 4
syscall

lw $a0, colsA
li $v0, 1
syscall

```

Picture 19: Function `input_done` (Part 1)

```

la $a0, matrixBText
li $v0, 4
syscall

lw $a0, rowsB
li $v0, 1
syscall

la $a0, xSymbol
li $v0, 4
syscall

lw $a0, colsB
li $v0, 1
syscall

la $a0, newline
li $v0, 4
syscall

j show_menu

```

Current dimensions:
Matrix A: 2 x 3
Matrix B: 3 x 2

Pictures 20 and 21: Function input_done (Part 2) and result after code execution

2.2.3. Display Matrix

In this part we going to explain how works display_matrices function

```

display_matrices:
    # Display matrix A
    la $a0, matrixAMsg # "Matrix A:"
    li $v0, 4
    syscall

    lw $s0, rowsA      # Load rowsA
    lw $s1, colsA      # Load colsA
    li $t0, 0          # row counter = 0

```

Picture 22: Display matrix A (Part 1)

We first start with displaying Matrix A, so we will print matrixAMsg, then load the values of row and column and set row counter to 0.

```

display_A_row:
    bge $t0, $s0, display_matrixB # if row >= rowsA, done
    li $t1, 0                     # col counter = 0

display_A_col:
    bge $t1, $s1, end_display_A_col # if col >= colsA, next row

    # Calculate address matrixA[row][col]
    mul $t2, $t0, $s1 # row * num_cols
    add $t2, $t2, $t1 # + col
    sll $t2, $t2, 2    # *4 (word size)
    la $t3, matrixA
    add $t3, $t3, $t2 # address of A[row][col]
    lw $a0, 0($t3)    # load value

    # Print value
    li $v0, 1
    syscall

    # Print space
    la $a0, space
    li $v0, 4
    syscall

    addi $t1, $t1, 1 # col++
    j display_A_col

end_display_A_col:
    # Print newline
    la $a0, newline
    li $v0, 4
    syscall

    addi $t0, $t0, 1 # row++
    j display_A_row

```

Picture 23: Display matrix A (Part 2)

If you pay attention you can see that procedure for input matrices and display matrices are the same, only difference is that we want now to display the value. Also the same procedure is for B matrix

```

display_matrixB:
    # Display matrix B
    la $a0, matrixBMsg # "Matrix B:"
    li $v0, 4
    syscall

    lw $s0, rowsB      # Load rowsB
    lw $s1, colsB      # Load colsB
    li $t0, 0          # row counter = 0

```

Picture 24: Display matrix B (Part 1)

```

display_B_row:
    bge $t0, $s0, display_done # if row >= rowsB, done
    li $t1, 0                  # col counter = 0

display_B_col:
    bge $t1, $s1, end_display_B_col # if col >= colsB, next row

    # Calculate address matrixB[row][col]
    mul $t2, $t0, $s1 # row * num_cols
    add $t2, $t2, $t1 # + col
    sll $t2, $t2, 2    # *4 (word size)
    la $t3, matrixB
    add $t3, $t3, $t2   # address of B[row][col]
    lw $a0, 0($t3)     # load value

    # Print value
    li $v0, 1
    syscall

    # Print space
    la $a0, space
    li $v0, 4
    syscall

    addi $t1, $t1, 1    # col++
    j display_B_col

end_display_B_col:
    # Print newline
    la $a0, newline
    li $v0, 4
    syscall

    addi $t0, $t0, 1    # row++
    j display_B_row

display_done:
    j show menu

```

Pictures 25 and 26: Display matrix B (Part 2) and function display_done

```

Matrix Operations Menu:
1. Enter matrices
2. Add matrices
3. Subtract matrices
4. Multiply matrices
5. Display matrices
6. Exit
Enter choice: 5

Matrix A:
1 2 3
4 5 6

Matrix B:
1 2
3 4
5 6

```

Picture 27: Demonstration of displaying matrices

2.2.4. Print Matrix

This function is the same as previous, so we just show the code, but procedure are the same

```

print_matrix:
    # Print result message
    li $v0, 4
    la $a0, resultMsg
    syscall

    # Initialize counters
    lw $s0, rowsA
    lw $s1, colsA
    li $t0, 0          # row counter

print_loop_row:
    bge $t0, $s0, end_print
    li $t1, 0          # col counter

print_loop_col:
    bge $t1, $s1, end_print_col

    # Calculate address
    mul $t2, $t0, $s1
    add $t2, $t2, $t1
    sll $t2, $t2, 2
    la $t3, matrixC
    add $t3, $t3, $t2

    # Print element
    li $v0, 1
    lw $a0, 0($t3)
    syscall

    # Print space
    li $v0, 4
    la $a0, space
    syscall

    addi $t1, $t1, 1
    j print_loop_col
end_print:
end_print_col:

```

Picture 28: Function print_matrix (Part 1)

```

end_print_col:
    # Print newline
    li $v0, 4
    la $a0, newline
    syscall

    addi $t0, $t0, 1
    j print_loop_row

end_print:
    jr $ra

```

Picture 29: Function print_matrix (Part 2)

2.2.5. Add Matrices

In this part we are going to talk about addition between two matrices

```

add_matrices:
    # Check matrix dimensions
    lw $t0, rowsA
    lw $t1, rowsB
    bne $t0, $t1, dimension_error
    lw $t0, colsA
    lw $t1, colsB
    bne $t0, $t1, dimension_error

    # Initialize counters
    lw $s0, rowsA
    lw $s1, colsA
    li $t0, 0                # row counter

```

Picture 30: Function add_matrices (Part 1)

First, we check the dimension of matrices, so first we load number of column of both matrices then compare. If there are equal, program will continue, if not then program will jump to dimension_error function. The same is for columns. After program check number of columns and rows, we load values for number of columns and rows of A matrix and set row counter to 0.

```

add_loop_row:
    bge $t0, $s0, end_add    # $s0 has row count
    li $t1, 0                # column counter

add_loop_col:
    bge $t1, $s1, end_add_col # $s1 has column count

    # Calculate address for matrixA[row][col]
    mul $t2, $t0, $s1        # row * num_cols
    add $t2, $t2, $t1        # + col
    sll $t2, $t2, 2          # multiply by 4 (word size)
    la $t3, matrixA
    add $t3, $t3, $t2         # address of A[row][col]
    lw $t4, 0($t3)           # value of A[row][col]

    # Calculate address for matrixB[row][col]
    la $t3, matrixB
    add $t3, $t3, $t2         # address of B[row][col]
    lw $t5, 0($t3)           # value of B[row][col]

```

Picture 31: Function add_matrices (Part 2)

Now, we will iterate through matrix. First we check number of row and row counter, if it is equal we jump to end_add function, if not then program will set column counter to 0 and enter to add_loop_col where we check number of column and column counter. If are equal, then program jump to end_add_col function, if not, then program will continue with execution. So first thing we get the value that we enter then according to this addressing formula we store the values: $t2 = (row * cols + col) * 4$ and this is the first three line of that part of code then we load base address add offset to base and store the value. Next, we repeat procedure for matrix B.

```

    # Add and store in matrixC
    add $t6, $t4, $t5
    la $t3, matrixC
    add $t3, $t3, $t2         # address of C[row][col]
    sw $t6, 0($t3)           # store result

    addi $t1, $t1, 1          # increment column counter
    j add_loop_col

end_add_col:
    addi $t0, $t0, 1          # increment row counter
    j add_loop_row

end_add:
    jal print_matrix
    j show_menu

```

Picture 32: Function add_matrices (Part 3)

The value that we stored from A and B matrix, we add and store to matrix C, then increment the column counter and repeat until the number of column and column counter are equal, then we repeat for rows and when we do all iterations we jump to print_matrix function to display matrix C. After this function program jump to show_menu function.

2.2.6 Subtract Matrices

Subtraction is the same procedure as addition, only exception is part when we add(in this case subtract) two numbers.

```
subtract_matrices:
    # Check matrix dimensions
    lw $t0, rowsA
    lw $t1, rowsB
    bne $t0, $t1, dimension_error
    lw $t0, colsA
    lw $t1, colsB
    bne $t0, $t1, dimension_error

    # Initialize counters
    lw $s0, rowsA
    lw $s1, colsA
    li $t0, 0                # row counter
```

Picture 33: Function subtract_matrices (Part 1)

```
sub_loop_row:
    bge $t0, $s0, end_sub    # $s0 has row count
    li $t1, 0                # column counter

sub_loop_col:
    bge $t1, $s1, end_sub_col # $s1 has column count

    # Calculate address for matrixA[row][col]
    mul $t2, $t0, $s1        # row * num_cols
    add $t2, $t2, $t1        # + col
    sll $t2, $t2, 2          # multiply by 4 (word size)
    la $t3, matrixA
    add $t3, $t3, $t2        # address of A[row][col]
    lw $t4, 0($t3)          # value of A[row][col]

    # Calculate address for matrixB[row][col]
    la $t3, matrixB
    add $t3, $t3, $t2        # address of B[row][col]
    lw $t5, 0($t3)          # value of B[row][col]
```

Picture 34: Function subtract_matrices (Part 2)

```

        # Subtract and store in matrixC
        sub $t6, $t4, $t5
        la $t3, matrixC
        add $t3, $t3, $t2    # address of C[row][col]
        sw $t6, 0($t3)      # store result

        addi $t1, $t1, 1    # increment column counter
        j sub_loop_col

    end_sub_col:
        addi $t0, $t0, 1    # increment row counter
        j sub_loop_row

end_sub:
    jal print_matrix
    j show_menu

```

Picture 35: Function subtract_matrices (Part 3)

2.2.7 Multiplication of matrices

So, as for addition/subtraction we check is possible to do a operation, then we we start with procedure.

```

multiply_matrices:
    # Check if colsA == rowsB
    lw $t0, colsA
    lw $t1, rowsB
    bne $t0, $t1, multiplication_error

    # Save original colsA
    lw $t9, colsA

    # Initialize counters
    lw $s0, rowsA    # $s0 = rowsA
    lw $s1, colsA    # $s1 = colsA (same as rowsB)
    lw $s2, colsB    # $s2 = colsB
    li $t0, 0        # i counter (rowsA)

```

Picture 36: Function multilplay_matrices (Part 1)

So to check is multiplication possible, we have to check is number of column of matrix A and number of row of matrix B are equal. After that we save the original number of column of matrix A and load number of rows and columns of matrix A and number of column of matrix B and set counter for rows of matrix A to 0.

```

mul_loop_i:
    bge $t0, $s0, end_mul
    li $t1, 0          # j counter (colsB)

mul_loop_j:
    bge $t1, $s2, end_mul_j
    li $t7, 0          # sum = 0
    li $t2, 0          # k counter (colsA/rowsB)

mul_loop_k:
    bge $t2, $s1, end_mul_k

    # Calculate A[i][k]
    mul $t3, $t0, $s1
    add $t3, $t3, $t2
    sll $t3, $t3, 2
    la $t4, matrixA
    add $t4, $t4, $t3
    lw $t5, 0($t4)     # A[i][k]

    # Calculate B[k][j]
    mul $t3, $t2, $s2
    add $t3, $t3, $t1
    sll $t3, $t3, 2
    la $t4, matrixB
    add $t4, $t4, $t3
    lw $t6, 0($t4)     # B[k][j]

```

Picture 37: Function mulitplay_matrices (Part 2)

So first we check number of rows of matrix A is equal to counter of rows of matrix A. If it is, we jump to end_mul function. If not, then we continue. Next step is to set another j counter to 0 then enter to antoher loop mul_loop_j . In this loop we again check number of columns of matrix B and j counter. If it is equal then jump to end_mup_j function. If is not, then set sum to 0 and k counter to 0. Then we enter through mul_loop_k function where we check is k counter equal to number of column of matrix A. If it is, program will jump to end_mul_k function. If not, then program will continue. So first thing we get the value that we enter then according to this addressing formula we store the values: $\$t2 = (\text{row} * \text{cols} + \text{col}) * 4$ and this is the first three line of that part of code then we load base address add offset to base and store the value. The same thing we will do with matrix B.

```

        # Multiply and accumulate
        mul $t8, $t5, $t6
        add $t7, $t7, $t8

        addi $t2, $t2, 1
        j mul_loop_k

    end_mul_k:
        # Calculate C[i][j] address
        mul $t3, $t0, $s2
        add $t3, $t3, $t1
        sll $t3, $t3, 2
        la $t4, matrixC
        add $t4, $t4, $t3
        sw $t7, 0($t4)

        addi $t1, $t1, 1
        j mul_loop_j

    end_mul_j:
        addi $t0, $t0, 1
        j mul_loop_i

```

Picture 38: Function multplay_matrices (Part 3)

So, we multiply values from matrix A and B then add to sum, increment counter k and repeat this process after counter k and number of column of matrix A are equal. Then we in **end_mul_k** the value sum will store to Matrix C and increment counter j, program will repeat all before until the counter j is equal to number of column of matrix B. In the **end_mul_j** function we just increment counter i until counter i is equal to number of rows of matrix A, then program jump to **end_mul** function

```

end_mul:
    # Print result with correct dimensions
    la $a0, resultMsg
    li $v0, 4
    syscall

    # Print using result dimensions (rowsA x colsB)
    lw $s0, rowsA    # Result rows
    lw $s1, colsB    # Result columns
    li $t0, 0        # row counter

```

Picture 39: Function multplay_matrices (Part 4)

When we fill matrix C then we display the result. The next code is the same as function `display_matrices`, just name of functions are different. So we do not explain more.

```
mul_print_loop_row:      # Changed label name
    bge $t0, $s0, end_mul_print
    li $t1, 0           # col counter

mul_print_loop_col:      # Changed label name
    bge $t1, $s1, end_mul_print_col

    # Calculate address
    mul $t2, $t0, $s1
    add $t2, $t2, $t1
    sll $t2, $t2, 2
    la $t3, matrixC
    add $t3, $t3, $t2

    # Print element
    li $v0, 1
    lw $a0, 0($t3)
    syscall

    # Print space
    li $v0, 4
    la $a0, space
    syscall

    addi $t1, $t1, 1
    j mul_print_loop_col
```

Picture 39: Function `multitplay_matrices` (Part 5)

```
end_mul_print_col:
    # Print newline
    li $v0, 4
    la $a0, newline
    syscall

    addi $t0, $t0, 1
    j mul_print_loop_row

end_mul_print:
    # Restore original colsA
    sw $t9, colsA
    j show_menu
```

Picture 40: Function `multitplay_matrices` (Part 6)

3. Conclusion

For this assignment, we made an assembly language program capable of performing basic matrix operations, showing how low-level programming can be employed in performing mathematical computations. The program successfully performed addition, subtraction, and multiplication operations on matrices of predefined dimensions, demonstrating assembly's potential in handling complex data structures. However, there lies room for improvement. The interface can be made more intuitive and user-friendly, and the current implementation can be made better by enabling dynamic matrices of any size rather than fixed dimensions. This would make the program more flexible, efficient, and handy for more general applications.

4. Resources

[MARS MIPS simulator](#)

<https://www.java.com/en/download/>

[Programmed Introduction to MIPS Assembly Language](#)

<https://youtube.com/playlist?list=PLyINWPMX1lPlmEeeMdbEFQo20eHAJL8hx&si=QpbKRQ-aB6XYnqIB>

https://youtube.com/playlist?list=PL5b07qImA3P6zUdDf-o97ddfpvPFuNa5A&si=PcfzoK58R_wbKiZt

<https://chatgpt.com/>

<https://www.deepseek.com/>

5. List of Figures

Picture 1: Matrix notation

Picture 2: How matrices multiplication works

Pictures 3 and 4: Matrices summation and subtraction

Picture 5: Declaring variables and reserve bytes in memory

Picture 6: Storing values

Picture 7: Messages

Picture 8: Text segment and main function

Picture 9: Function show_menu

Picture 10: Function exit_program

Picture 11: Functions dimension_error and multiplication_error

Picture 12: Input matrix for matrix A (Part 1)

Picture 13: Input matrix for matrix A (Part 2)

Picture 14: Result after code execution

Picture 15: Input matrix for matrix A (Part 3)

Picture 16: Input matrix for matrix B (Part 1)

Picture 17: Input matrix for matrix B (Part 2)

Picture 18: Input matrix for matrix B (Part 3)

Picture 19: Function input_done (Part 2)

Pictures 20 and 21: Function input_done (Part 2) and result after code execution

Picture 22: Display matrix A (Part 1)

Picture 23: Display matrix A (Part 2)

Picture 24: Display matrix B (Part 1)

Picture 25 and 26: Display matrix B (Part 2) and function display_done

Picture 27: Demonstration of displaying matrices

Picture 28: Function print_matrix (Part 1)

Picture 29: Function print_matrix (Part 2)

Picture 30: Function add_matrices (Part 1)

Picture 31: Function add_matrices (Part 2)

Picture 32: Function add_matrices (Part 3)

Picture 33: Function subtract_matrices (Part 1)

Picture 34: Function subtract_matrices (Part 2)

Picture 35: Function subtract_matrices (Part 3)

Picture 36: Function mulitplay_matrices (Part 1)

Picture 37: Function mulitplay_matrices (Part 2)

Picture 38: Function mulitplay_matrices (Part 3)

Picture 39: Function mulitplay_matrices (Part 4)

Picture 40: Function mulitplay_matrices (Part 5)